Towards Continuous-Time Structure from Motion: An Investigation of Temporal Bias in Event-Based SfM

Mohamed Jacer Aloui
Robotic Interactive Perception
Technische Universität Berlin
Berlin, Germany
Supervisor: Prof. Guillermo Gallego

***Abstract*—This report details the development of a custom Structure from Motion (SfM) pipeline designed for asynchronous event cameras. We utilize the SuperEvent framework [1] and Multi-Channel Time Surfaces (MCTS) to bridge the gap between asynchronous event streams and classical geometric solvers. Unlike standard approaches that rely on external libraries, we implement all the SfM components from scratch, including multi-view geometry as well as optimization and robust estimation. Our experiments show that while event-based SfM works well under constant speeds, varying kinematics introduce a "feature lag" that causes structural decay. We analyze this lag empirically and propose future architectural changes to mitigate depth errors in dynamic scenarios. Code available at: github.com/jacerdev/ev_sfm**

***Index Terms*—Event Cameras, Structure from Motion, Multi-Channel Time Surfaces, Feature Extraction, Asynchronous Sensors.**

I. Introduction

Most Structure from Motion (SfM) pipelines are designed for standard frame-based cameras. This works well under normal conditions, but these cameras often struggle with severe motion blur at high speeds or fail entirely in bad lighting. Event cameras offer a potential solution because they only record pixel intensity changes asynchronously, providing high dynamic range (HDR) and microsecond resolution. The main issue, however, is matching features between views. Standard event-based tracking methods often fail in cluttered or fast-moving environments, while reliable SfM depends on finding very stable keypoints.

To fix this matching problem, we used **SuperEvent**, a data-driven feature extractor, to process the event streams for our SfM pipeline. Since we don't have large manually labeled datasets for event cameras, SuperEvent is trained using self-supervision, learning from standard camera frames that were recorded alongside the events. Instead of passing the raw asynchronous events directly into the pipeline, we feed the network **Multi-Channel Time Surfaces (MCTS)**. By separating positive and negative pixel changes into different channels and using multiple time windows, MCTS creates a much more stable representation of the scene, regardless of how fast the camera is moving.

The main goal of this project was to take these deep-learned event features and integrate them directly into an SfM pipeline that we built completely from scratch. We wanted to see if we could plug modern event-based feature extraction into classical multi-view geometry math to successfully reconstruct 3D environments from asynchronous camera data.

II. Background

A. Geometric Foundations of Camera Models

The fundamental mechanism of Structure from Motion (SfM) relies on the **pinhole camera model**. This model is the intuitive baseline for understanding how a camera sees the world, establishing a projective mapping from the 3D world $\mathbb{R}^3$ to the 2D image plane $\mathbb{R}^2$. If we consider a 3D point $P = [x, y, z]^T$, it is projected onto the image plane through the center of projection O. This results in a 2D point $P' = [f\frac{x}{z}, f\frac{y}{z}]^T$, where f denotes the focal length of the lens.

Because this transformation involves a division by the depth coordinate z, it is non-linear. To linearize this process and make it computationally solvable, we use **homogeneous coordinates**. By appending a "1" to the end of our vectors, we can represent the projection as a linear matrix-vector product. The complete mapping is written in the **camera projection matrix** $P = K[R|t]$. This matrix has 11 degrees of freedom. The **intrinsic matrix** K accounts for internal properties: focal length, the principal point offset, and pixel skew. The **extrinsic parameters**, which consist of a rotation matrix R and a translation vector t, define exactly where the camera is located and its orientation.

B. Epipolar Geometry and Multi-View Constraints

Reconstructing a 3D structure from 2D images is inherently difficult because depth information is lost during projection. We resolve this ambiguity by relying on **epipolar geometry** between different views. Fundamentally, if we observe a point in one image, we know it must lie along a specific "epipolar line" in the second image.

The algebraic relationship between corresponding points p and p' in two views is governed by the **Fundamental matrix** F for uncalibrated cameras, or the **Essential matrix** $E =$

$[t_\times]R$ for calibrated ones. To calculate this, we implement the **Normalized Eight-Point Algorithm**. We must normalize the pixel coordinates first; otherwise, the system of equations becomes numerically unstable and yields highly inaccurate results. By solving this linear system, we can recover the camera's relative motion between two time steps.

C. Reconstruction and Bundle Adjustment

Once the camera motion is estimated, we must calculate the 3D position of the tracked points. **Triangulation** is the process of estimating a 3D point from its 2D projections across multiple views. While we can use a linear method like Singular Value Decomposition (SVD), it is highly sensitive to noise. Therefore, we follow it with a non-linear optimization—specifically the **Levenberg-Marquardt** (LM) algorithm. The goal of LM is to minimize the "reprojection error," which is the spatial distance between the actual 2D observation and where our computed 3D model projects onto the image plane.

The final comprehensive step is **Bundle Adjustment (BA)**. This is a large-scale optimization process. Instead of fixing points individually, it jointly refines all camera poses and all 3D points simultaneously. By doing this, we ensure the entire 3D map is globally consistent and geometrically rigid.

D. Event Representation and Time Surfaces

Standard multi-view geometry operates on synchronous frames, where all pixels are captured at the exact same instant. Event cameras operate differently; they are fundamentally asynchronous. Each event consists of a pixel coordinate (x, y), a precise timestamp t, and a polarity p. To process this within our SfM architecture, we must first "structure" the event stream.

We utilize **Multi-Channel Time Surfaces (MCTS)**. By applying a temporal decay function, we accumulate these raw asynchronous events into 2D tensors that behave similarly to images, recording the recency of events at each pixel. See Fig. 1

Crucially, our downstream mathematical solvers still operate under a **synchronous assumption**. We assume that all features extracted within an MCTS window occurred simultaneously at the exact end-timestamp of that window.

III. System Architecture: Bridging Events and Classical Geometry

For this project, we decided against using standard libraries like OpenCV for the core math. Instead, we implemented the geometrical solvers from scratch so we could fully understand and control how they interact with the event data.

A. Bootstrapping the Map

To start building the 3D map, we need a baseline between two initial views. We take the first two MCTS frames, get the keypoint matches from SuperEvent, and calculate their relative pose using Epipolar RANSAC. This helps us find a good Essential matrix while ignoring bad matches. See Fig. 2

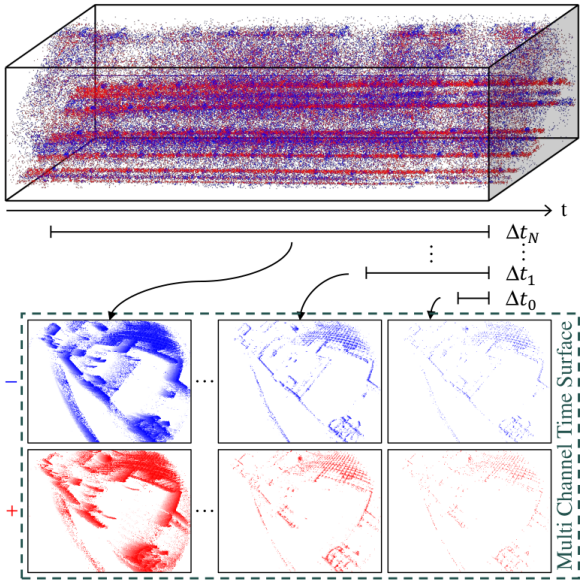


Fig. 1: Conversion of raw asynchronous event streams into structured Multi-Channel Time Surfaces using logarithmic decay.

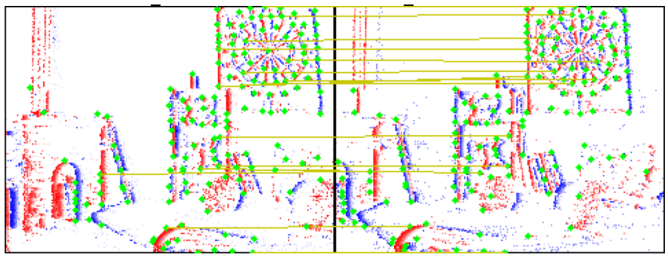


Fig. 2: Feature matching between the first two MCTS frames using SuperEvent. These initial matches serve as the baseline for computing the Essential matrix.

We compute the Essential matrix using the 8-point algorithm and apply a Sampson distance threshold to validate the matches. SVD is then used to decompose the matrix into rotation and translation components. We apply a "cheirality check" to select the one out of four possible solutions that places the triangulated 3D points physically in front of both cameras. Once the initial points are anchored, we refine them using our custom LM optimizer, which relies on a hand-calculated **analytic Jacobian**.

B. Tracking and Incremental Expansion

With an initial map established, we continuously track the camera as it moves. For every new MCTS, we solve the Perspective-n-Point (PnP) problem by matching new 2D features to known 3D structural points. We use a linear Direct Linear Transformation (DLT) to generate an initial pose guess, which is then fed into our LM-based PnP loop utilizing analytic Jacobians. As the camera explores new areas, we triangulate the newly observed 2D-2D matches to expand the 3D point cloud.

C. Graph Maintenance and Bundle Adjustment

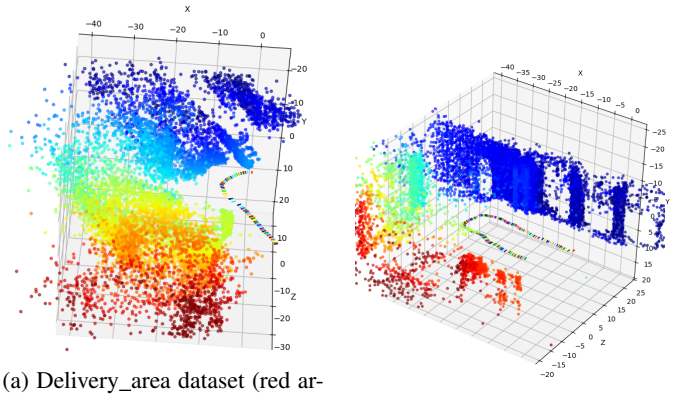
To keep the computational load manageable, we implement a ”Keyframe” strategy. A frame is only promoted to a Keyframe if it observes sufficient new geometry. To combat scale drift, where small estimation errors compound over time, we periodically run a **Local Bundle Adjustment**. This isolates a window of recent Keyframes and the 3d points the observe and optimizes them jointly, pulling the map back into geometric consistency.

IV. Experiments and Results

A. Frame-Based Reconstruction Baseline

We first tested the pipeline on the standard ‘delivery_area‘ and ‘electro_rig‘ datasets of ETH3D. This provided a necessary baseline to verify that our custom SfM implementations were mathematically sound before introducing event data.

- On delivery_area, we tracked 60 keyframes and reconstructed nearly 12,000 points with a Root Mean Square Error (RMSE) of just 0.75 px. See Fig. 3
- On electro_rig, we successfully registered 153 keyframes and 11,500 points with an RMSE of 0.84 px. See Fig. 3



(a) Delivery_area dataset (red arrow shows starting camera position).

(b) Electro_rig dataset (red arrow shows starting camera position).

Fig. 3: Reconstruction results on two frame-based datasets.

B. Event-Based Reconstruction

We then evaluated the pipeline utilizing the MCTS features extracted from event streams.

Linear Trajectories (‘slider_depth‘): In this sequence, the camera is mounted on a mechanical slider moving laterally. The pipeline performed quite well. We tracked 10 keyframes yielding 472 points. The generated 3D map correctly isolated structural depth, and the computed camera path formed a straight line that precisely mirrored the physical hardware constraint (RMSE: 1.55 px). See Fig. 4

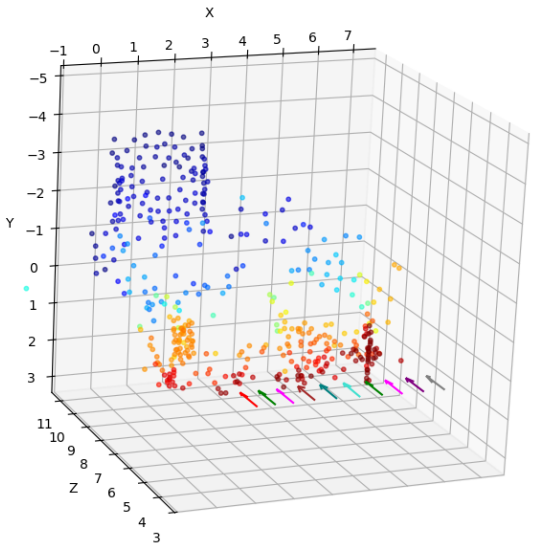


Fig. 4: 3D reconstruction of the ‘slider_depth‘ sequence. The calculated trajectory matches the physical hardware constraint. (red arrow shows starting camera position)

Non-Linear Motion Contexts (‘urban‘): This scenario presents a greater challenge as the camera moves backward (we processed the sequence in reverse to guarantee initial feature gradients, as the dataset otherwise starts with frames with very few events). We successfully tracked 18 keyframes and roughly 1,000 3D points. While sparser, the primary vertical gradients of the building are distinctly visible in the 3D output (RMSE: 2.09 px). See Fig. 5

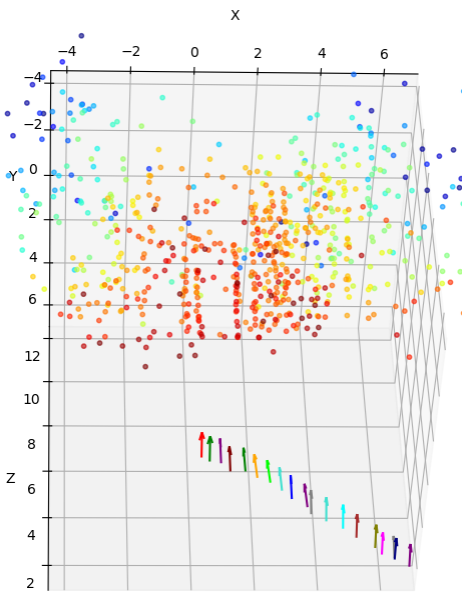


Fig. 5: 3D reconstruction of the ‘urban‘ sequence. The calculated trajectory matches the physical hardware constraint. (top view of the scene. red arrow shows starting camera position)

C. Structural Failures and Limitations (‘office_spiral‘)

During the ‘office_spiral‘ sequence, the camera moves in a continuous circular loop. Although our 2D feature tracking remained stable, the resulting 3D structure collapsed entirely (RMSE: 4.45 px). This failure occurs because the circular motion provides exceptionally narrow triangulation baselines, pushing the projective geometry solvers beyond their operational limits. See Fig. 6

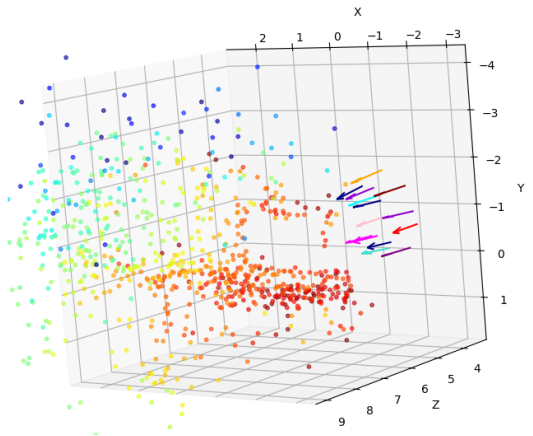


Fig. 6: 3D reconstruction of the ‘office_spiral‘ sequence. The calculated trajectory are somewhat plausible. It’s also possible to make out the screen frame on the desk. (red arrow shows starting camera position)

D. Theoretical Analysis: Asynchronous Feature Lag

Standard SfM solvers operate on a synchronous assumption, where every feature in a given frame is treated as having been captured at the exact same instant t_i. For event-based representations like MCTS, this is physically inaccurate as the network extracts features from a temporal integration window ΔT. This discrepancy introduces a *feature-specific temporal lag* δt, where the perceived 2D position is actually the camera’s state at $t_{obs} = t_{end} - \delta t$ (if $\delta t = 0$ then the feature was captured exactly at the last instant of the integration window which is the assumption). When this lag is inconsistent across matching views, the solver incorrectly interprets the temporal sub-pixel shift as spatial geometric disparity, leading to systematic triangulation bias. We categorize the resulting structural deformations into three cases:

- **Consistent Lag ($\delta t_1 \approx \delta t_2$):** The relative temporal shift is preserved across views. The absolute epipolar disparity remains correct, yielding accurate depth estimation, though the entire 3D track suffers a minor lateral translation along the direction of motion.
- **Lag Divergence ($\delta t_1 > \delta t_2$):** The feature in the first view is “older” relative to its temporal window. This artificially *inflates* the measured disparity between the frames, causing the solver to underestimate distance—a phenomenon we identify as *depth shrinkage*.
- **Lag Convergence ($\delta t_1 < \delta t_2$):** The feature in the second view is older, resulting in a compressed disparity. The triangulation solver compensates for this lack of separation by overestimating the distance, leading to *depth expansion*.

This lag is visible in Fig. 7, showing SuperEvent keypoints overlaid on the UZH `urban` sequence. The camera moves backward and slightly right, so vertical building edges should shift rightward. However, the keypoints of the left vertical border stay to the left of the actual edge. Note this border is furthest from the camera which means it has least disparity compared to the other vertical borders here, and that’s why the model seemingly only ’understood’ the border through long channels. This confirms the network ’sees’ an older state from the temporal window. For this visualization, we use the 4th channel of the MCTS, as it serves as a sweet spot: while the lower-index channels (shorter τ) are frequently too sparse, the 5th channel suffers from integrated low-contrast. As mentioned, the MCTS generation utilizes a logarithmic spread of decay constants $\tau \in [0.001, 0.003, 0.01, 0.03, 0.1]$ seconds. A simple fix would be using only short integration windows, but this prevents the network from extracting complex semantic features, which require longer event histories to correctly ’understand’ the scene.

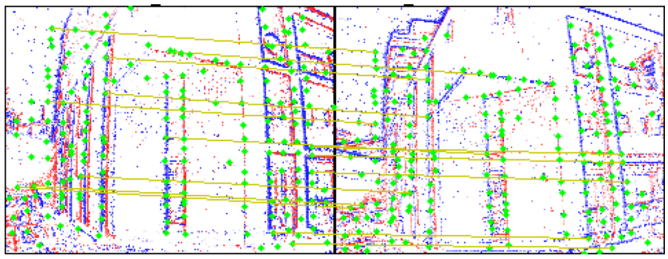
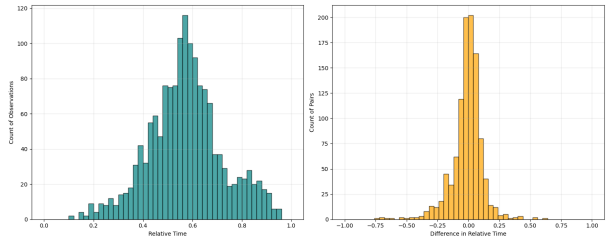
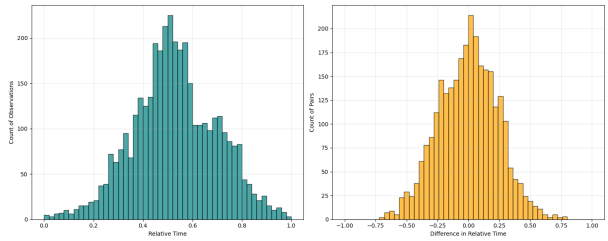


Fig. 7: Example from ‘urban‘ dataset showing mismatch between some features’ extracted position and actual current pixel positions)

To analyze the systematic temporal bias, we utilize the statistical distribution of feature timestamps within the MCTS windows. The relative timestamp (0.0 to 1.0) indicates where a feature sits within its specific temporal window, while the pairwise difference measures the change in this relative position between every 2 matched features.



(a) Slider_depth sequence: the histogram of pairwise differences tightly constrained Δt distribution (Mean: -0.012, Std: 0.128), indicating strong temporal consistency.



(b) Urban sequence: the histogram of pairwise differences has broader Δt distribution (Mean: -0.012, Std: 0.243), reflecting higher temporal variance due to dynamic motion.

Fig. 8: Temporal consistency diagnostics of MCTS feature extractions. The pairwise chronological shift (Δt) of tracked 3D points between consecutive frames shows that constant kinematics (top) maintain tight temporal alignment, while dynamic kinematics (bottom) introduce significant temporal dispersion, leading to depth compression artifacts.

1) **Slider Data:** Because the velocity was constant, the feature lag remained constant. The average pairwise shift between frames was practically zero (-0.012). Consequently, the 3D map remained structurally accurate.
2) **Urban Data:** The erratic velocity in this sequence caused the lag shift variance to swell to 0.243. This empirically confirms that dynamic kinematics severely violate the ”synchronous frame” assumption, causing structural decay.

V. Conclusion and Future Work

In this project, we successfully built an SfM pipeline from scratch that works directly with data-driven event descriptors. Our results show that while the system is highly robust when the camera moves at a constant speed, varying speeds introduce a feature lag issue that degrades the 3D reconstruction.

Future Work: The depth issues caused by these mismatched time channels show that we might need to rethink how event networks output their features. One potential solution for future work is a **Mixture of Experts** approach. In this setup, the long MCTS channels would be used only to understand what the feature actually is (the *semantic descriptor*), while the shortest, most recent channels would be used strictly to find its *exact pixel location*. Additionally, if the neural network could output a precise timestamp for every single feature it detects, classical SfM pipelines could easily correct for temporal lag during bundle adjustment, completely avoiding the heavy computational cost of continuous-time optimization.

References

[1] D. Gehrig, A. Loquercio, and D. Scaramuzza, “SuperEvent: Cross-Modal Learning of Event-based Keypoint Detection for SLAM,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2023.